

TRABAJO TERMINAL II

FlowCode: Compilador de Diagramas de Flujo a Código C para Dispositivos Móviles

Número del TT: 2026-A038

Grupo: 8CM1

Entregable 1 Ciclo 4: Motor de Análisis

Presenta:

García Quiroz Gustavo Ivan

Profesor de seguimiento del TT:

Prof. Edgardo Adrián Franco Martínez

Introducción al Motor de Análisis (Ciclo 4)

El Motor de Análisis constituye la fase central del conversor FlowCode. En este ciclo se desarrolló el mecanismo encargado de extraer la estructura lógica y sintáctica de los diagramas de flujo proporcionados por el usuario, preparándolos para su posterior traducción a código C. A continuación, se detallan los módulos principales que integran este motor, referenciando los archivos de código correspondientes.

1. Análisis Léxico (Archivo: `lib/compiler/lexical_analyzer.dart`)

Se implementó un analizador encargado de examinar el texto de cada nodo en el diagrama de flujo de forma secuencial. El proceso consiste en agrupar los caracteres en unidades lógicas con significado, conocidas como lexemas o "tokens" (cuya estructura se define en `lib/compiler/token.dart`).

Durante esta fase, se reconocen 91 tipos de tokens organizados en categorías fundamentales: palabras reservadas en español equivalentes al lenguaje C (como "entero", "si", "mientras"), identificadores de variables, literales numéricas y de cadena, así como operadores aritméticos, relacionales y lógicos. Este proceso descarta elementos prescindibles, como espacios en blanco, y construye la primera abstracción del contenido ingresado.

2. Análisis Sintáctico (Archivo: `lib/compiler/syntax_analyzer.dart`)

Una vez extraídos los tokens, se desarrolló un analizador sintáctico basado en la técnica de descenso recursivo predictivo. Este módulo verifica que la secuencia de lexemas cumpla estrictamente con las reglas de una gramática formal predefinida.

Por ejemplo, en un nodo de asignación, la gramática valida la presencia obligatoria de un identificador, seguido de un operador de igualdad y una expresión válida. Como resultado de esta fase, se construye el Árbol de Sintaxis Abstracta (AST, implementado en `lib/compiler/ast_nodes.dart`), el cual organiza las instrucciones en una jerarquía que refleja claramente la precedencia de operadores y el flujo lógico del programa.

3. Análisis Semántico (Archivo: `lib/compiler/semantic_analyzer.dart`)

Posteriormente, se implementó el análisis semántico con el propósito de asegurar la coherencia lógica de las instrucciones expresadas en el AST. Se verificó la consistencia en el uso de los identificadores, comprobando rigurosamente que toda variable haya sido declarada previamente.

Asimismo, se integraron verificaciones de compatibilidad de tipos en operaciones y asignaciones (por ejemplo, validando que operaciones aritméticas se apliquen únicamente sobre operandos numéricos). Adicionalmente, se incluyeron mecanismos de advertencia

para detectar variables no inicializadas y variables declaradas pero no utilizadas, previniendo comportamientos indefinidos.

4. Tabla de Símbolos (Archivo: lib/compiler/symbol_table.dart)

Para dar soporte al análisis semántico, se diseñó e integró la Tabla de Símbolos, actuando como la estructura de datos central del motor de conversión. Este componente almacena los metadatos de todos los identificadores (variables, constantes y parámetros) descubiertos durante el análisis.

En esta tabla se registra el nombre, el tipo de dato subyacente y la ubicación exacta de la declaración (nodo y línea). La gestión de la tabla permite realizar búsquedas rápidas durante la evaluación de expresiones y sienta las bases para el manejo de ámbitos de alcance local y global en la arquitectura del sistema.

5. Sistema de Errores (Archivo: lib/compiler/compiler_errors.dart)

Finalmente, se estructuró un sistema unificado para el manejo de errores. Este módulo captura las anomalías identificadas en cualquiera de las fases previas (léxica, sintáctica o semántica) y genera retroalimentación precisa al usuario.

El sistema clasifica las incidencias mediante un código y un nivel de severidad (informativo, advertencia, error o fatal). Además de localizar el nodo exacto que originó el fallo, proporciona mensajes descriptivos y sugerencias de corrección en idioma español, facilitando el aprendizaje y la resolución iterativa de los diagramas.

Evidencias de Integración en la Aplicación

A continuación, se exponen tres plantillas algorítmicas clásicas visualizadas en la aplicación FlowCode. Para ilustrar a profundidad el funcionamiento del Motor de Análisis, se desglosa el proceso de validación mostrando la interfaz principal junto con los resultados específicos de las fases de análisis para cada diagrama.

Plantilla '12. Factorial Iterativo'

- **Evidencia 1.1 - Diagrama Base:** Se ilustra el diagrama del cálculo de un factorial mediante ciclos. En esta captura se espera observar la carga del algoritmo iterativo en el lienzo principal del editor.

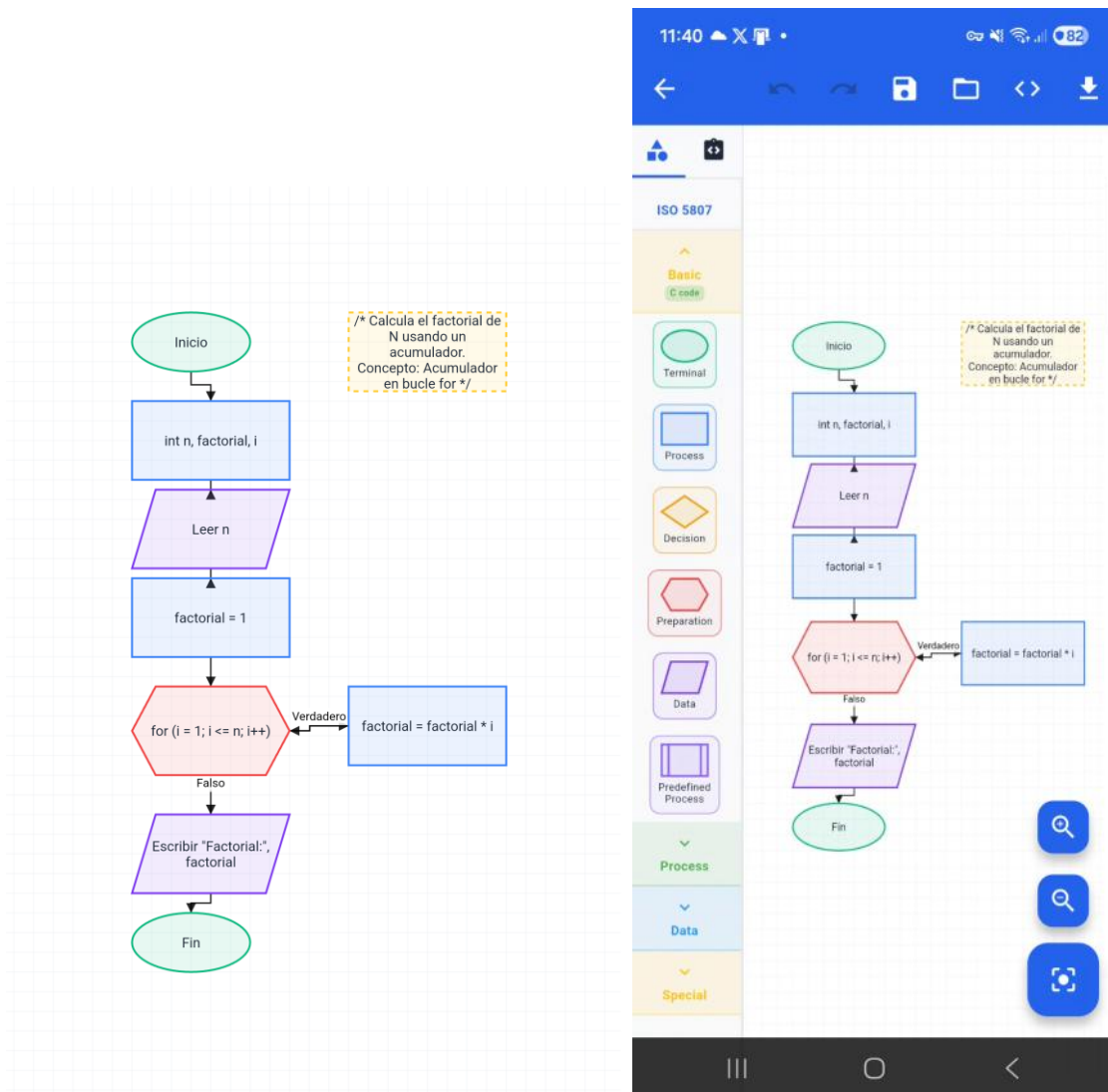
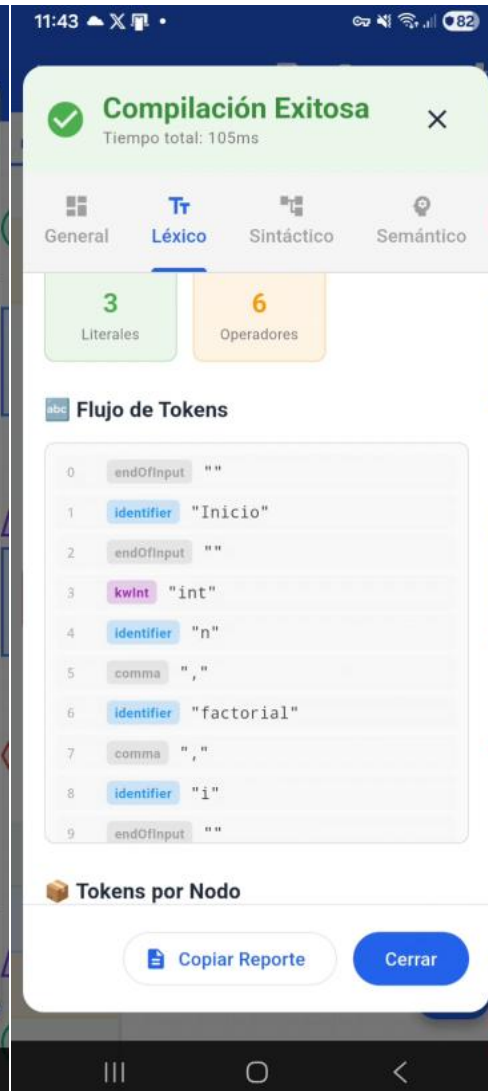
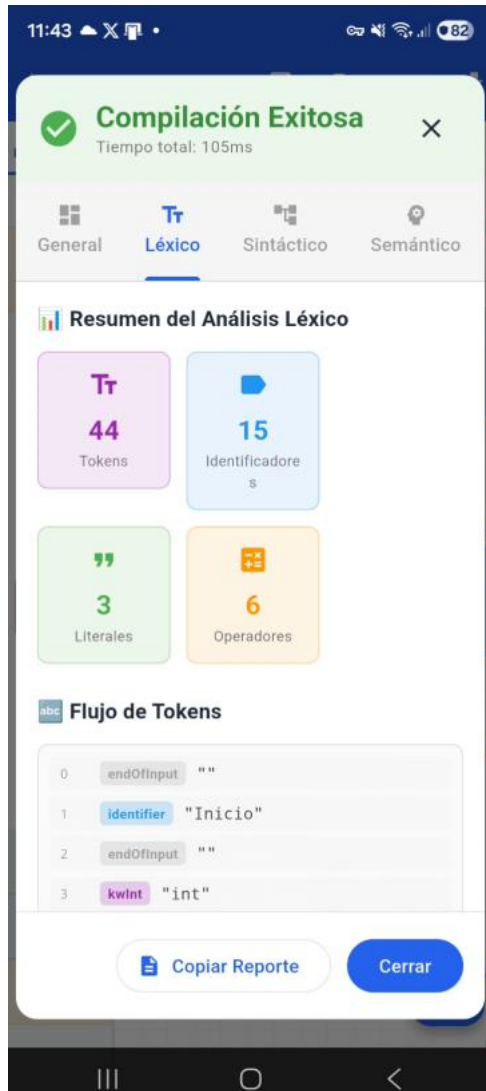
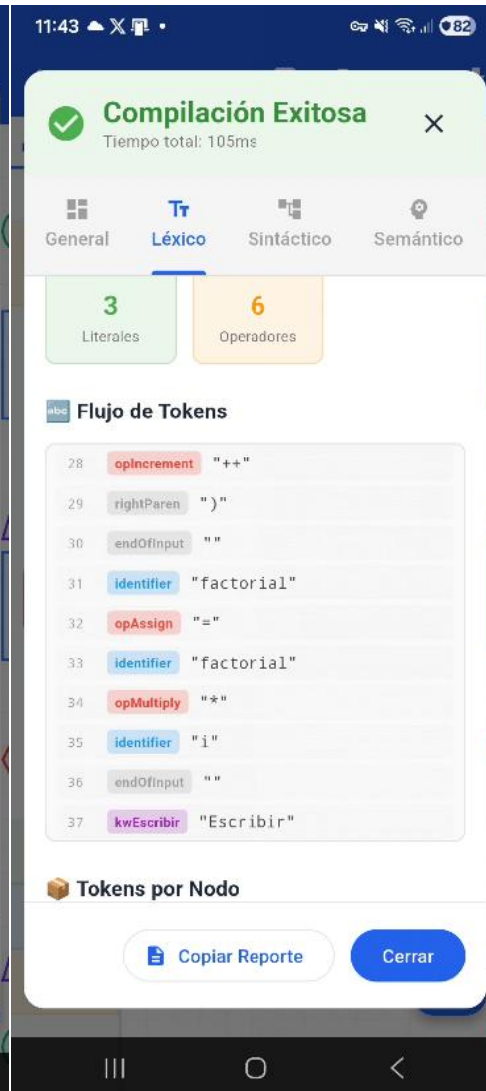


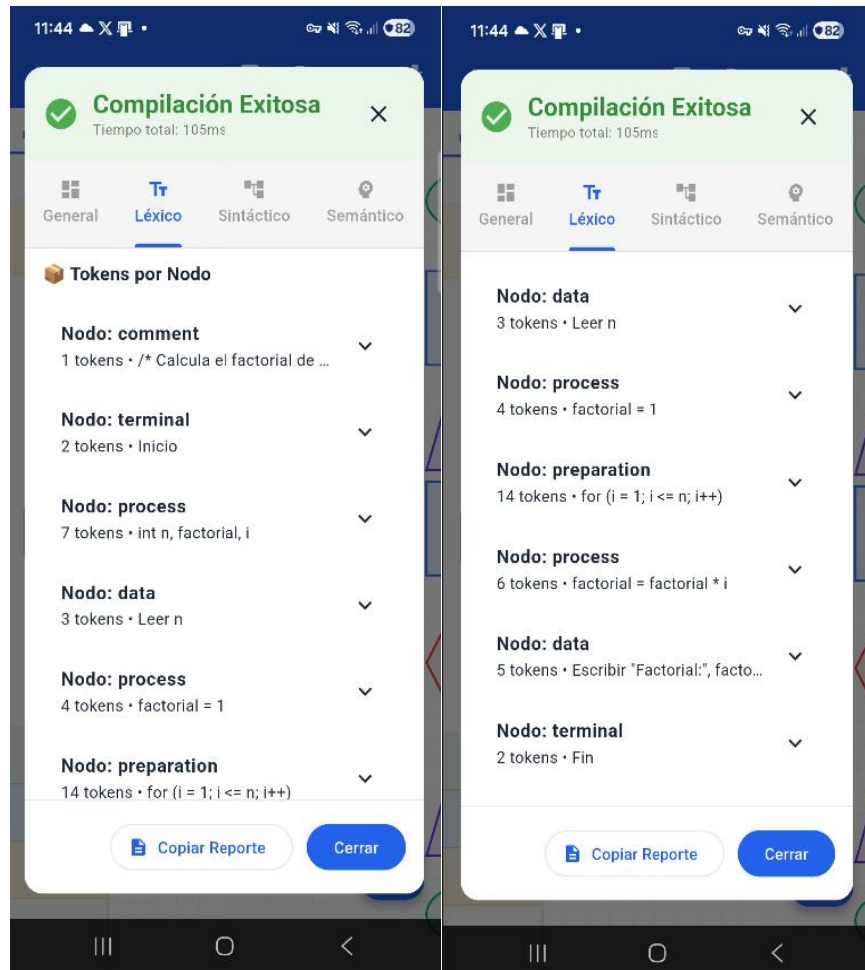
DIAGRAMA FACTORIAL ITERATIVO

- **Evidencia 1.2 - Análisis Léxico:** Se expone el resultado del escáner léxico. Aquí se aprecia cómo el motor agrupa las entradas del usuario en lexemas, distinguiendo

palabras reservadas, números y los operadores matemáticos de acumulación del factorial.

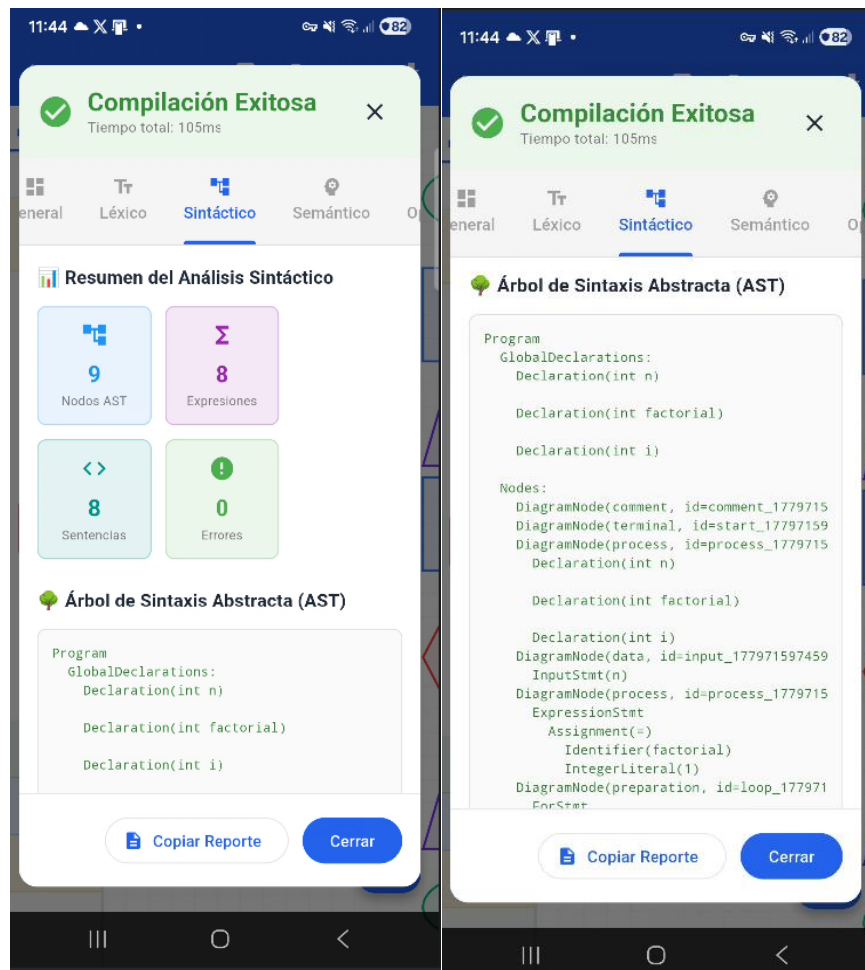


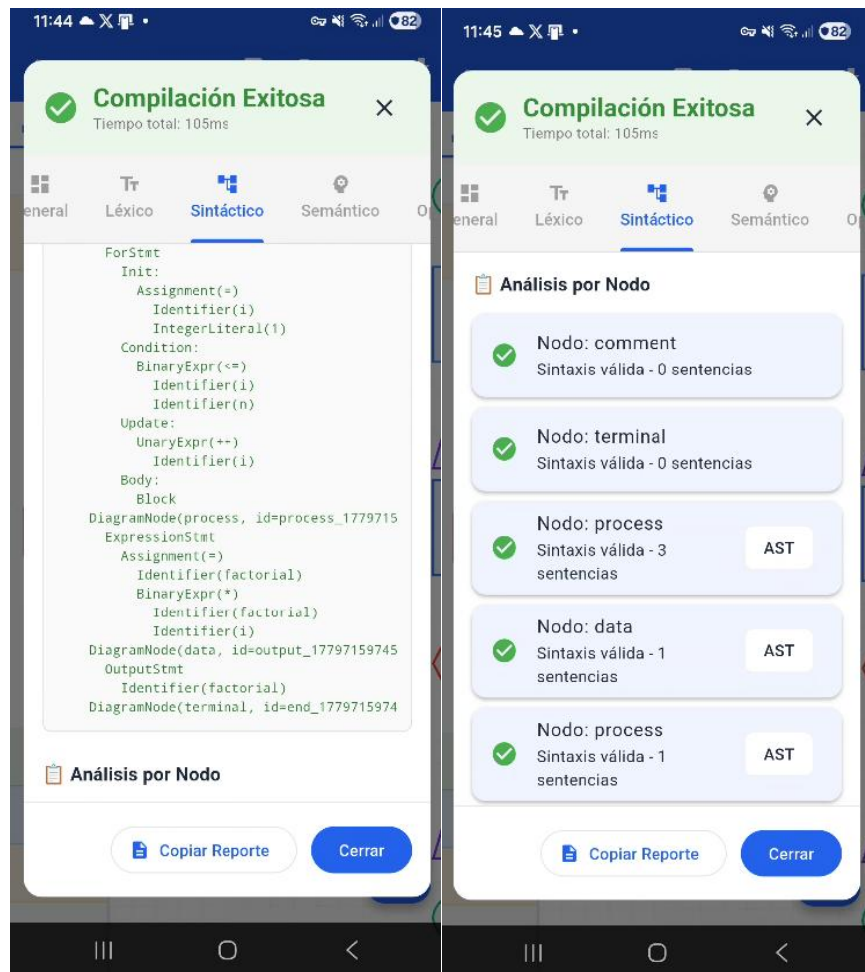


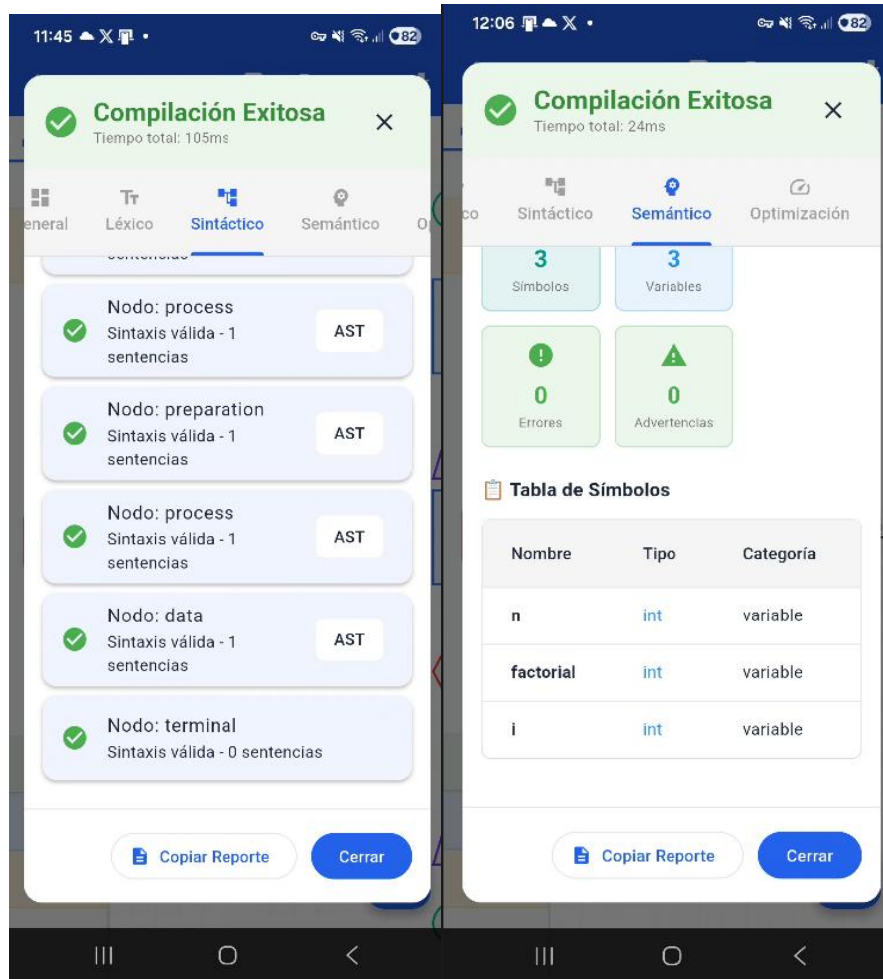


RESULTADOS DE ANÁLISIS LÉXICO (TOKENS)

- **Evidencia 1.3 - Análisis Sintáctico y Semántico:** Se visualizan las comprobaciones de gramática (AST) y coherencia semántica. Se evidencia la validación de las sentencias de control y el estado de la Tabla de Símbolos sin errores para las variables de iteración.



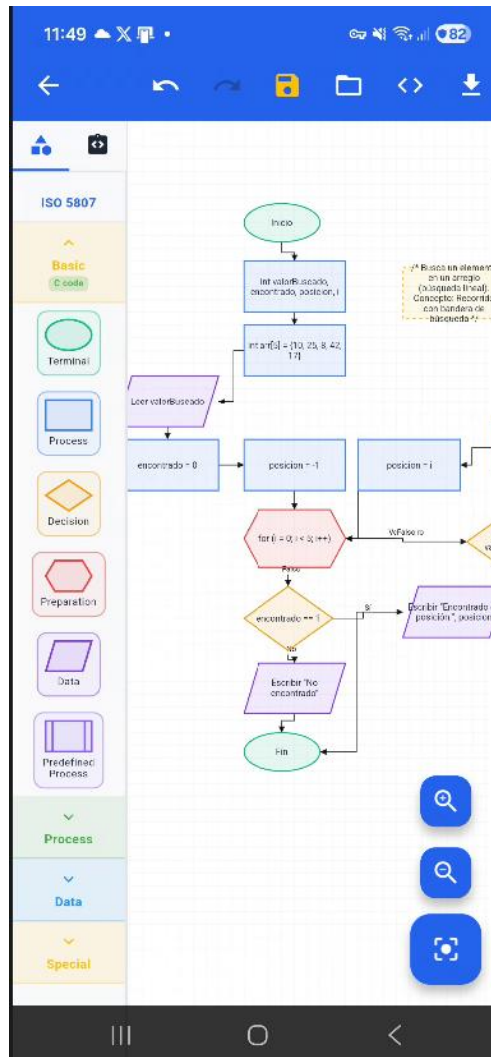




RESULTADOS AST Y TABLA DE SIMBOLOS

Plantilla '14. Búsqueda Secuencial'

- **Evidencia 2.1 - Diagrama Base:** Se expone la estructura visual de la búsqueda secuencial en arreglos, mostrando el manejo de nodos condicionales e iterativos en la aplicación.



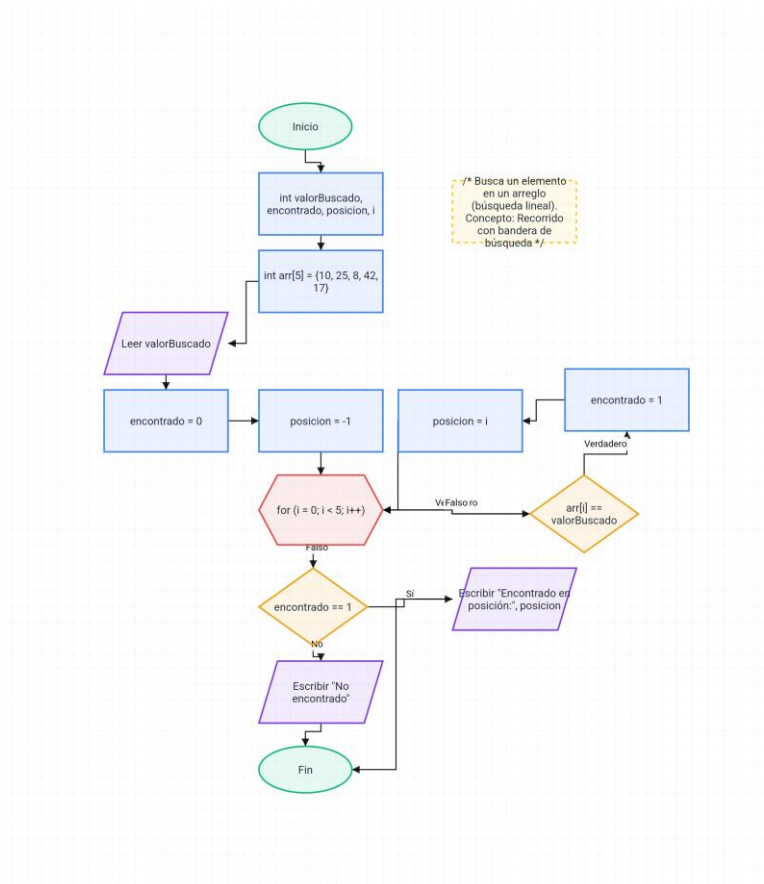
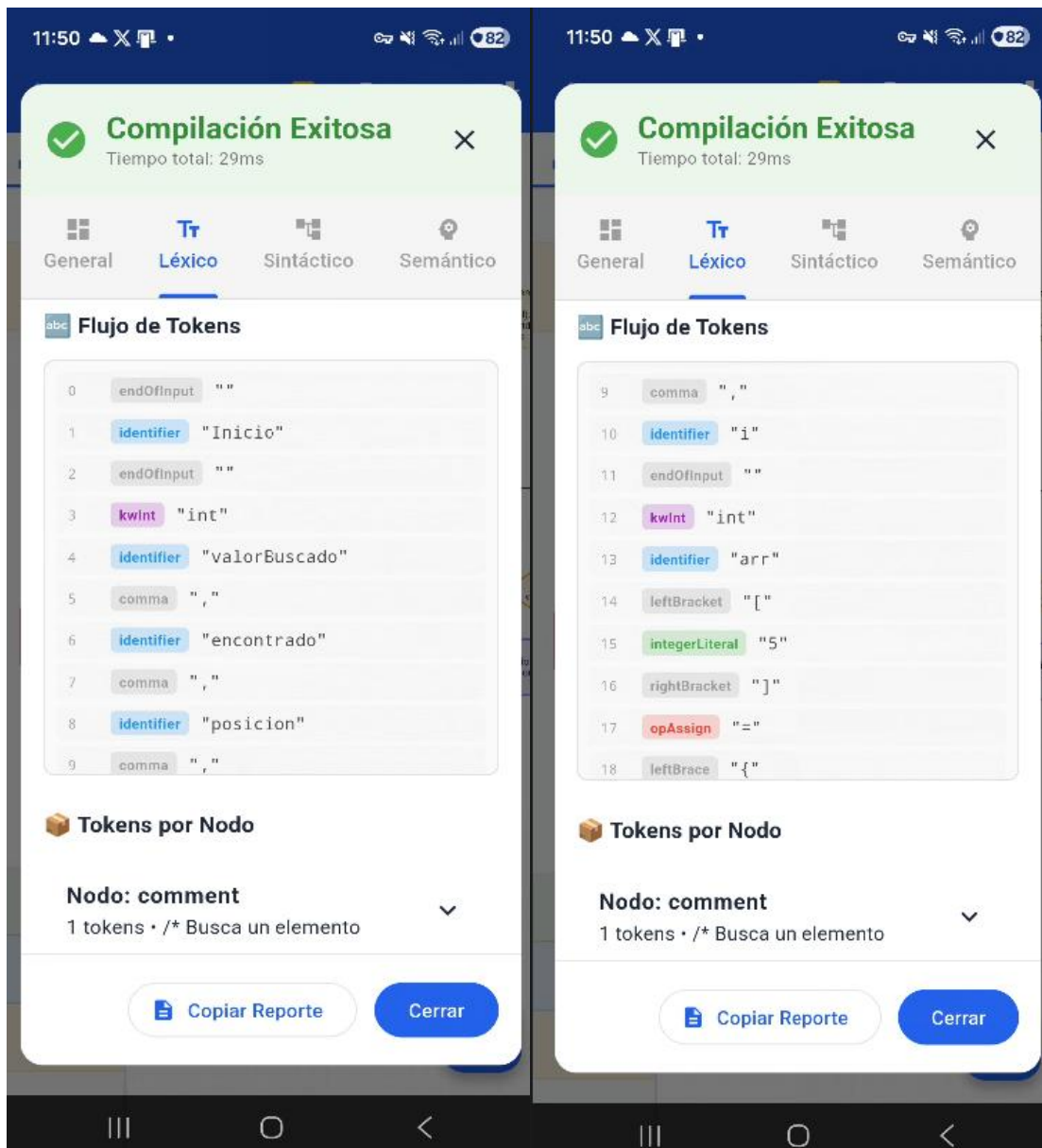
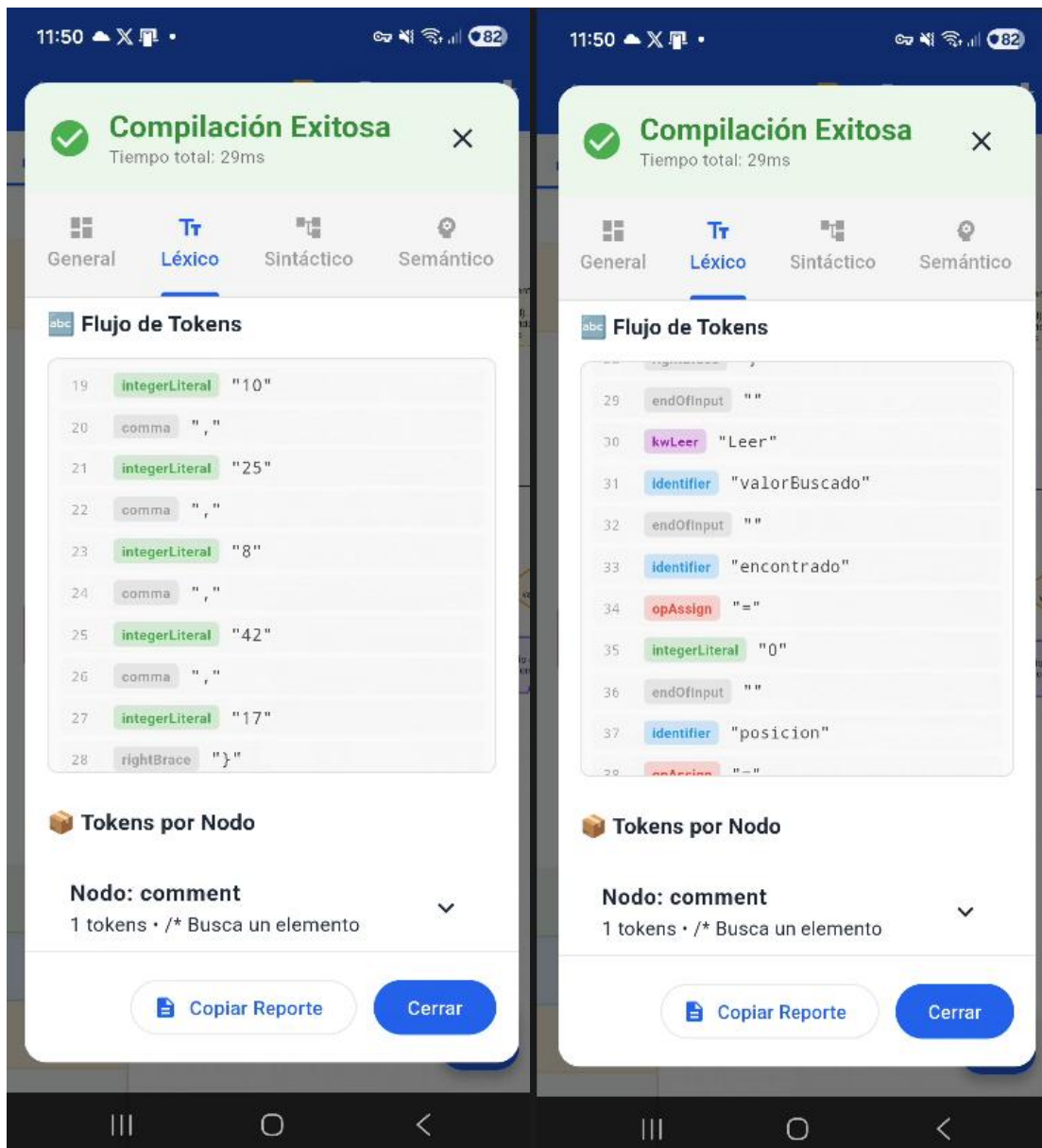
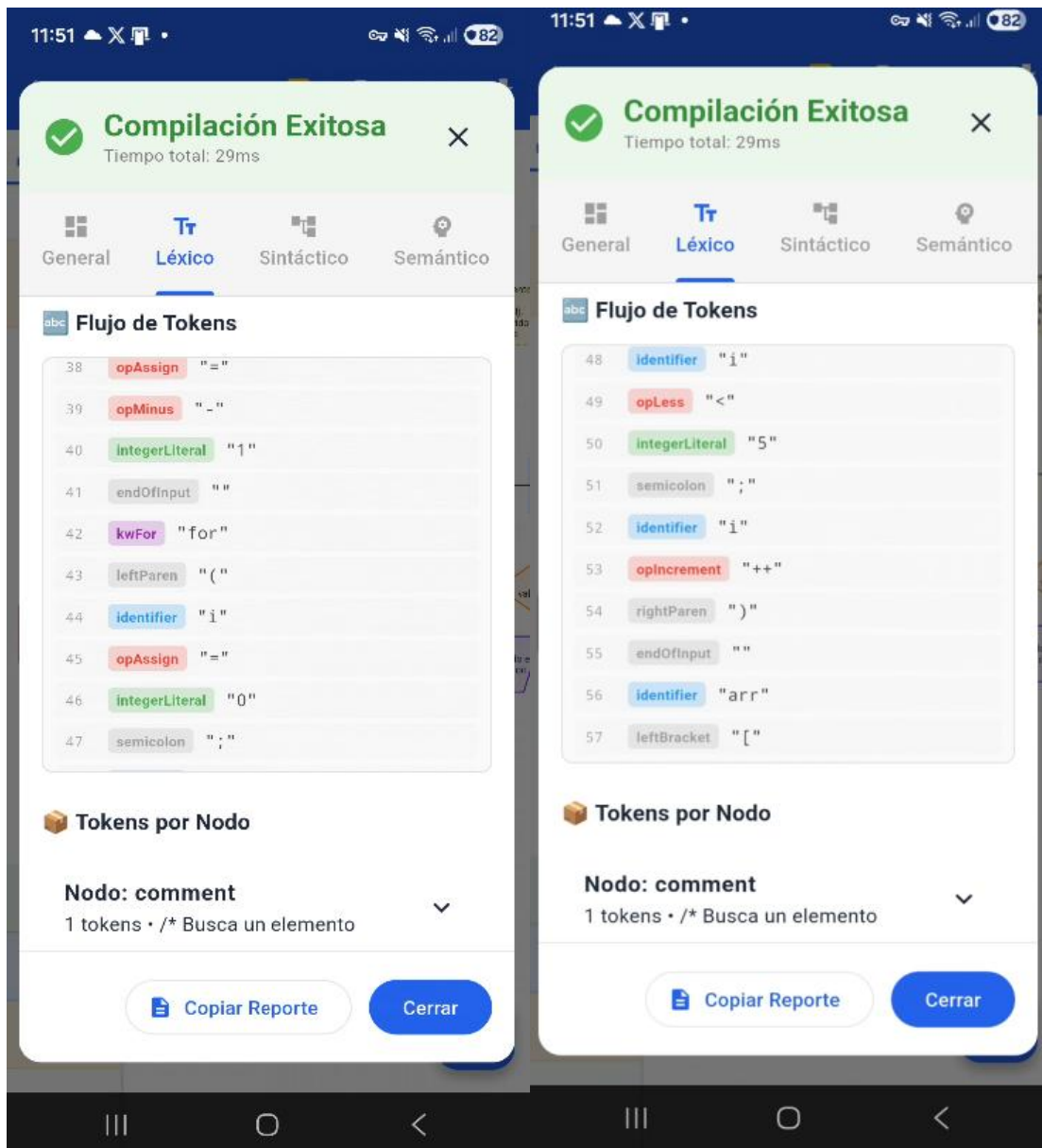


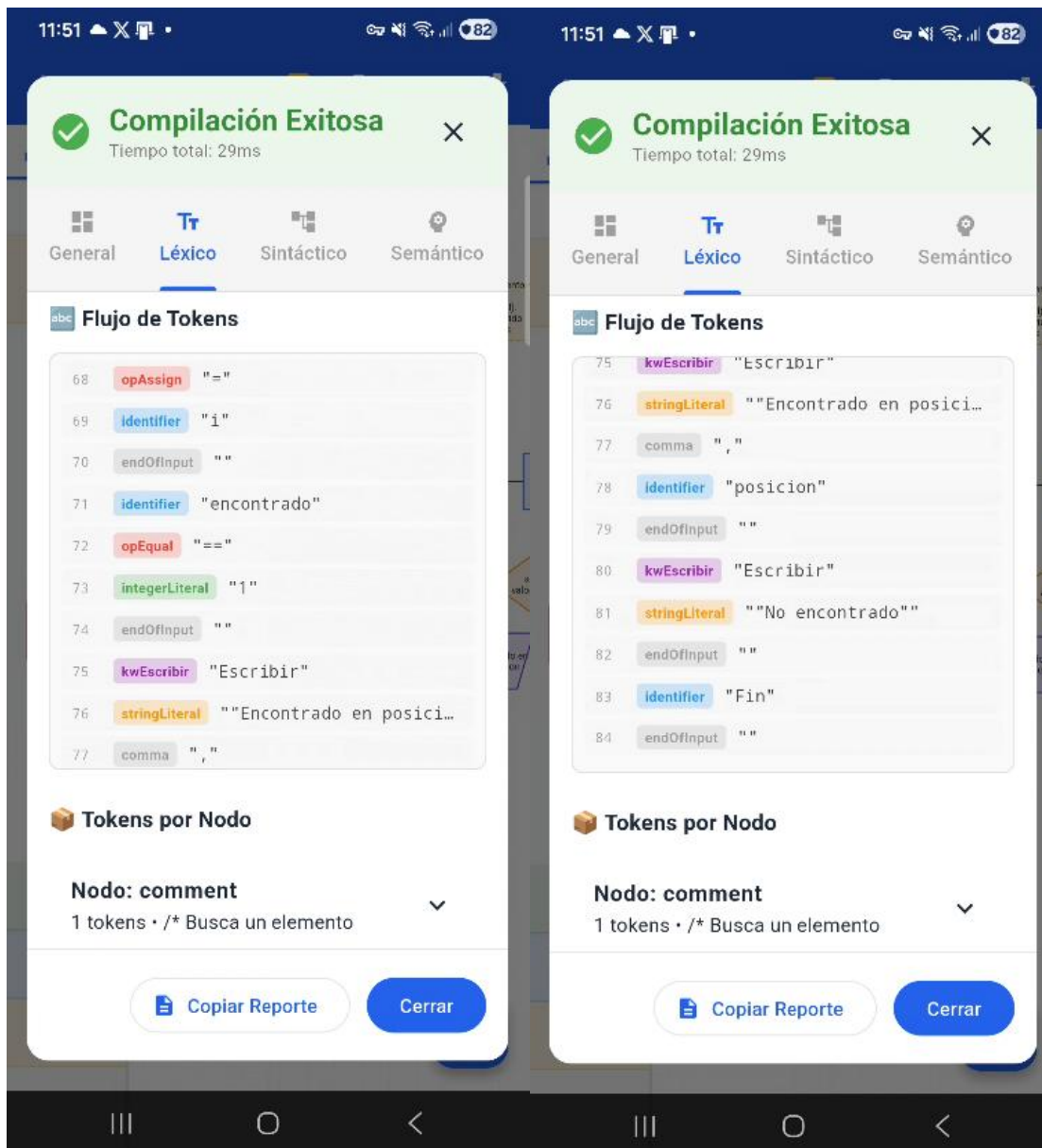
DIAGRAMA BÚSQUEDA SECUENCIAL

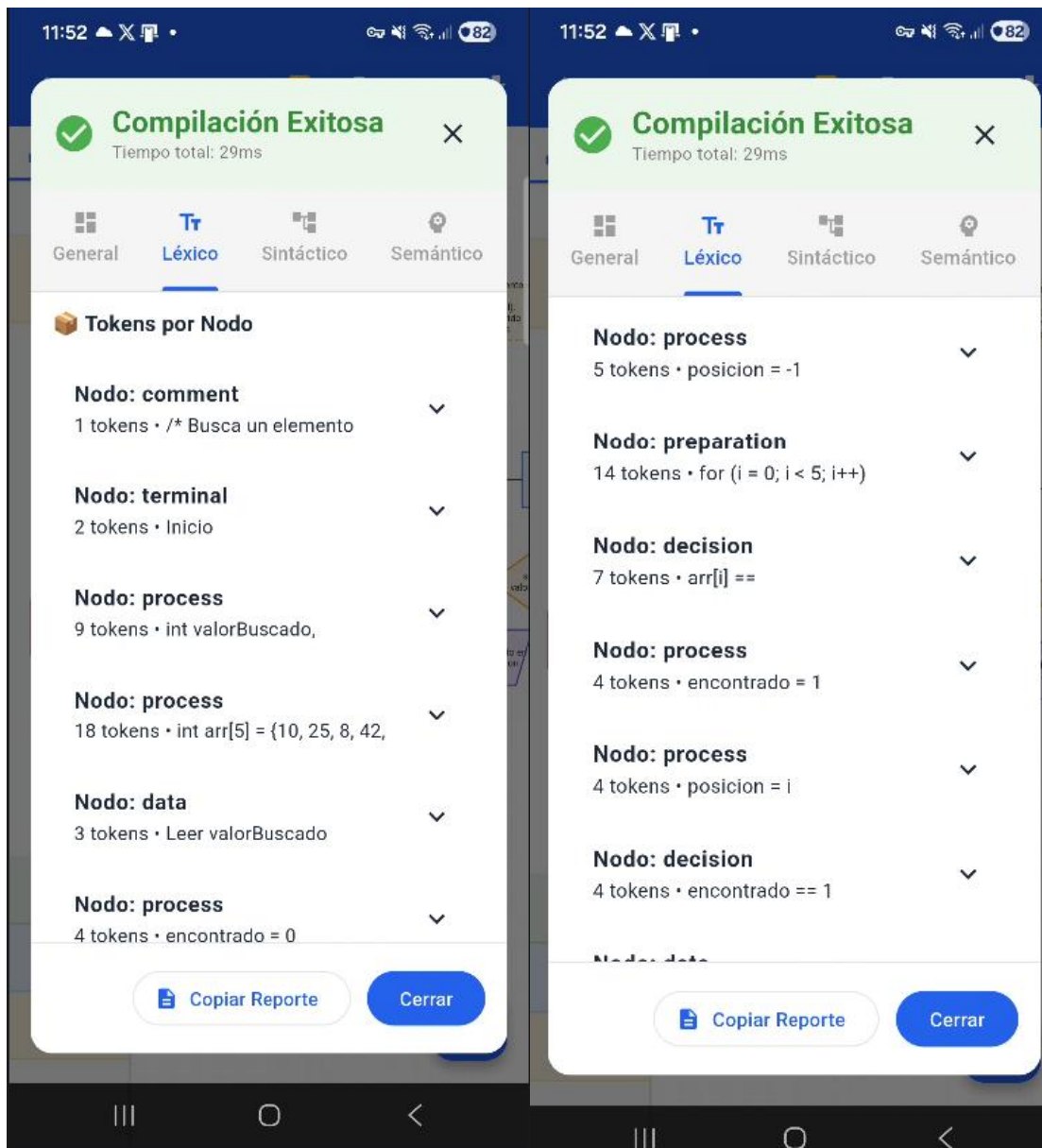
- **Evidencia 2.2 - Análisis Léxico:** Se evidencia el reconocimiento de identificadores para arreglos y variables de índice, comprobando la robustez de la fase de tokenización en la extracción de expresiones lógicas complejas.

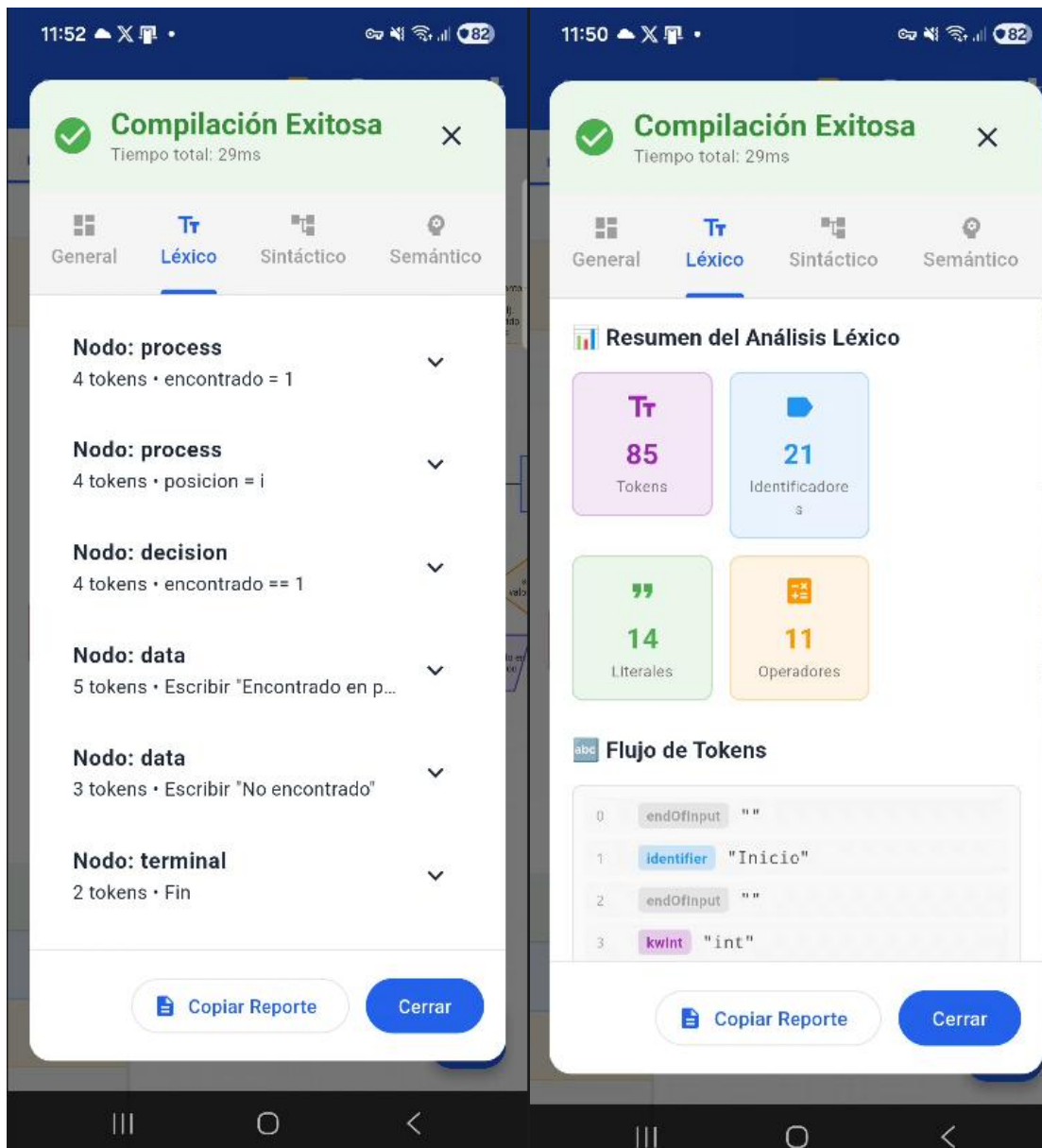






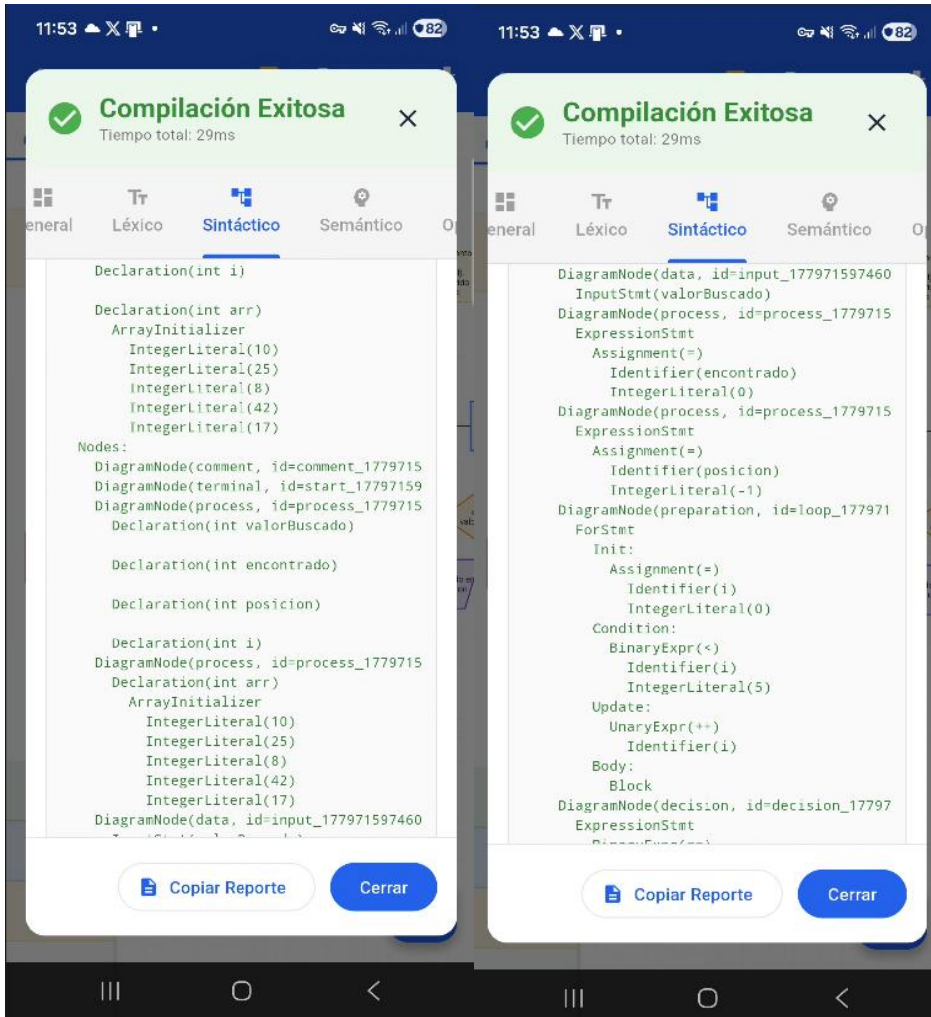


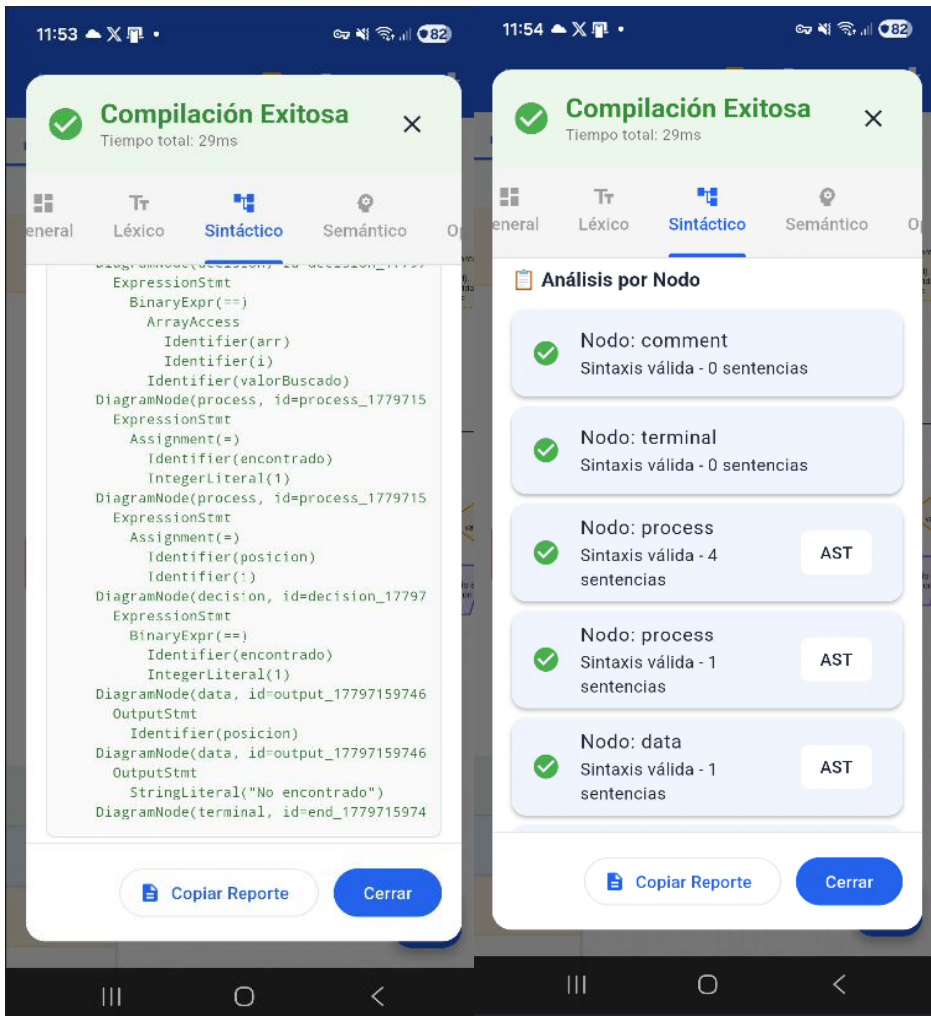


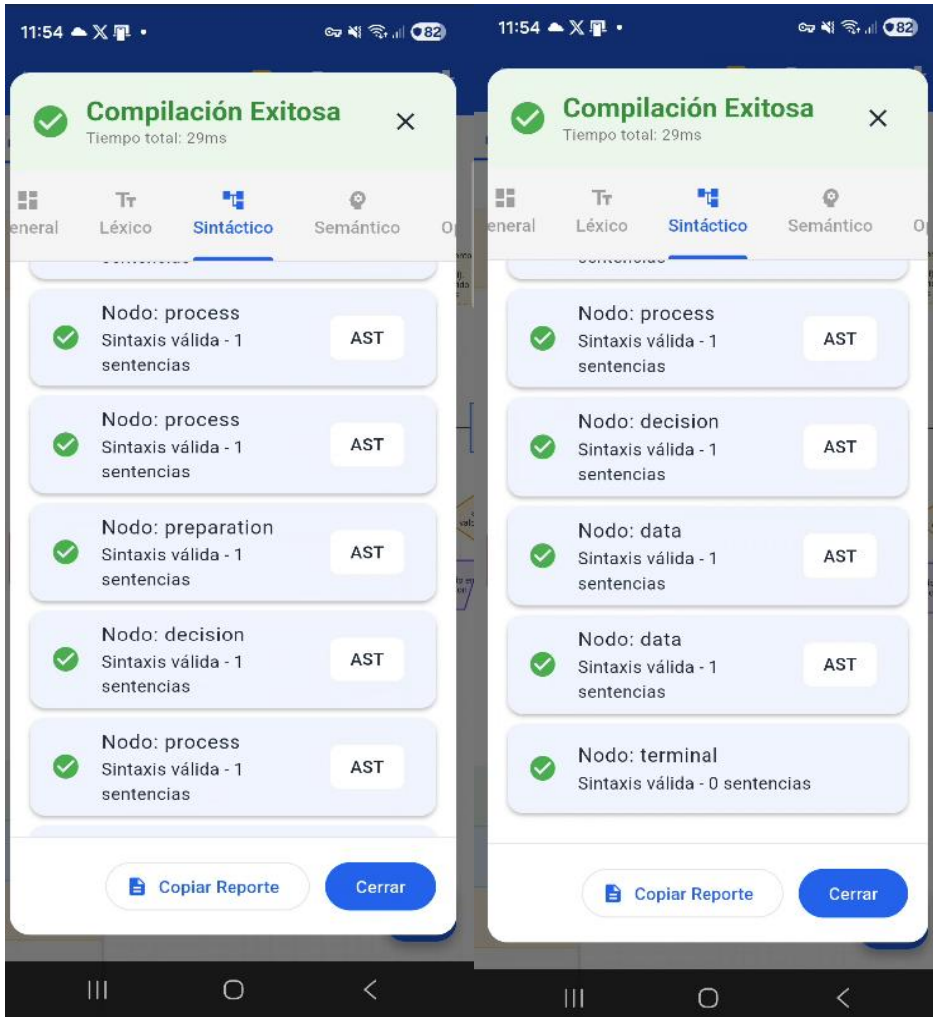


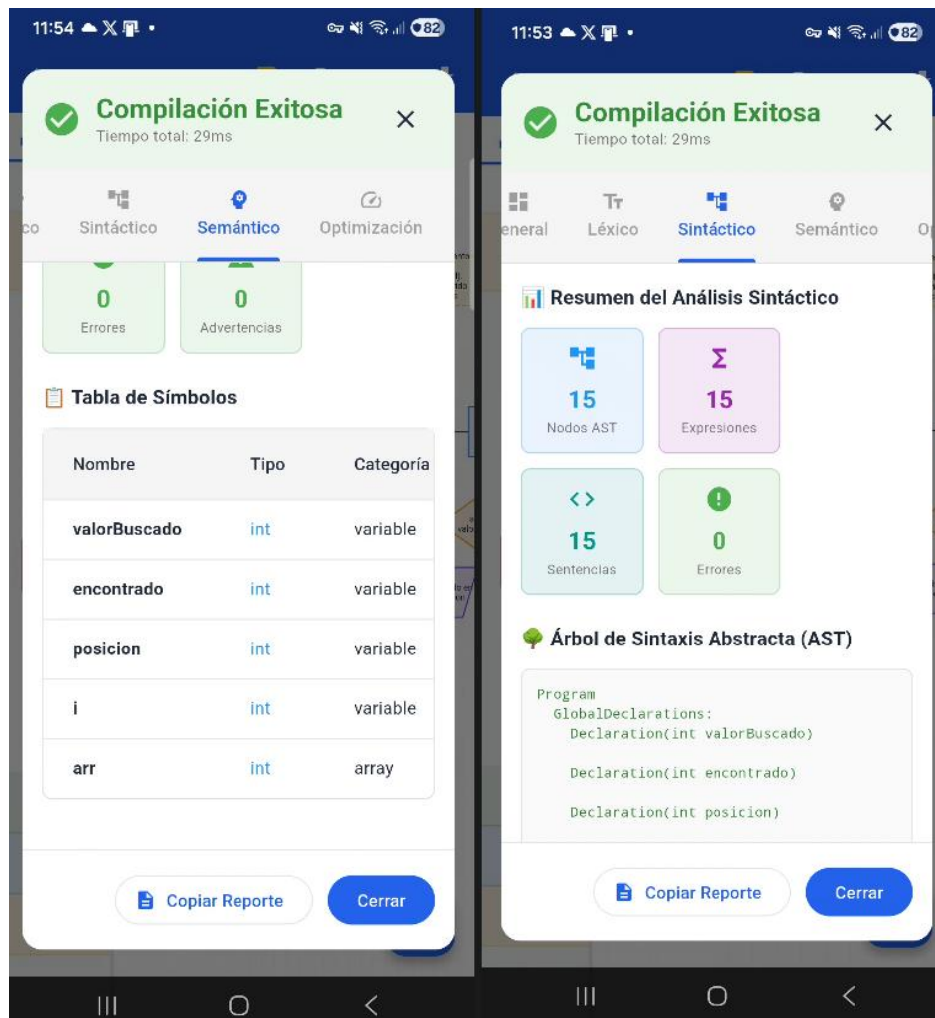
RESULTADOS DE ANÁLISIS LÉXICO (TOKENS)

- **Evidencia 2.3 - Análisis Sintáctico y Semántico:** Se muestran los resultados del validador. La imagen constata la verificación gramatical del acceso a arreglos y la validación semántica de tipos en las sentencias lógicas de igualdad.









RESULTADOS AST Y TABLA DE SÍMBOLOS